

Reviewer Pack — Non-Backtracking Spectral Gap Bounds Tseitin Resolution Leng...

Entry a58cbdc6e05 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-16 21:44:20 UTC

Non-Backtracking Spectral Gap Bounds Tseitin Resolution Length

Entry ID: a58cbdc6e05 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-16 21:44:20 UTC

1. Conjecture

Field A (mathematical branch): Non-backtracking (Hashimoto) operator spectrum on graphs **Field B** (complexity object): Resolution refutation length / KW-refutation game depth for Tseitin formulas

Statement:

Let G be a connected d -regular graph ($d \geq 3$) on n vertices with odd-total Tseitin charge σ , let $B(G)$ be the $2m \times 2m$ Hashimoto non-backtracking edge operator with spectral radius $\rho(B)$, and define the graph invariant $\nu(G) := n \cdot \max(0, \log((d-1)/\rho(B)))$. We conjecture (i) every Resolution refutation π of $T(G, \sigma)$ satisfies $|\pi| \geq 2^{c \cdot \nu(G)}$ for an absolute constant $c \geq 1/16$, and equivalently (via Pudlák's KW-refutation game) the refutation game has communication cost $\geq c \cdot \nu(G)$; (ii) $\nu(G) = O(1)$ for every graph that is not a vertex expander (any G with a balanced cut of size $o(n)$ has $\rho(B) \rightarrow d-1$, hence $\nu \rightarrow 0$). A single Tseitin instance refuted by Resolution in fewer than $2^{c \cdot \nu(G)}$ steps refutes the conjecture.

Rationale (proposer's reasoning):

The Hashimoto/Ihara non-backtracking spectrum captures geometric-group-theoretic cogrowth and Ramanujan-style expansion finer than the ordinary Laplacian, yet is computed by a single polynomial-time eigen-decomposition. KW-refutation games convert Resolution refutations into edge-charge-flow games on G whose communication cost is bounded by precisely the non-backtracking mixing rate of charges along geodesics, so $\nu(G)$ is the natural information-flow witness for the BSW expansion-vs-width principle. The invariant is structural (graph-only) rather than truth-table single-number, sidesteps Natural Proofs, and uses no polynomial extension of the Boolean ring, so it avoids Algebrization.

Taxonomy category: KARCHMER_WIGDERSON (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): c0722d24bc746643

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across 30 seeds $\times n \in \{6, 8, 10, 12, 16\}$, three families (a) Ramanujan-like 3-regular, (b) glued non-expanders, (c) C_n : (1) OLS slope of $\log_2 R$ on v for family (a) > 0.25 ; (2) every family-(b),(c) instance has $v < 1.5$ AND $\log_2 R < 1.5n$; (3) no instance with $v \geq 0.5n$ has $\log_2 R < 0.1 \cdot v$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.95	SAFE	SAFE
ALGEBRIZATION	SAFE	0.90	SAFE	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 3 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - Tseitin formulas resolution lower bounds expansion - non-backtracking Hashimoto spectral radius Ramanujan expander graphs - resolution refutation size Tseitin expander spectral gap

Top relevant hits considered: - [<http://arxiv.org/abs/2209.05839v3>] On bounded depth proofs for Tseitin formulas on the grid; revisited - [<http://arxiv.org/abs/2103.09609v1>] Characterizing Tseitin-formulas with short regular resolution refutations - [<http://arxiv.org/abs/2601.13117v2>] The Case for Cardinality Lower Bounds

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, $\leq 90s$ wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import sys
import random
import math
import itertools
from collections import defaultdict

def matrix_mult(A, B):
    n = len(A)
    m = len(B[0])
    result = [[0 for _ in range(m)] for _ in range(n)]
    for i in range(n):
        for j in range(m):
            for k in range(len(B)):
                result[i][j] += A[i][k] * B[k][j]
    return result

def matrix_transpose(A):
    return [list(row) for row in zip(*A)]

def matrix_norm(A):
```

```

    return max(sum(abs(x) for x in row) for row in A)

def power_iteration(A, max_iter=100, tol=1e-6):
    n = len(A)
    b = [random.random() for _ in range(n)]
    b_norm = math.sqrt(sum(x**2 for x in b))
    b = [x / b_norm for x in b]

    for _ in range(max_iter):
        b_new = [0.0] * n
        for i in range(n):
            for j in range(n):
                b_new[i] += A[i][j] * b[j]

        b_new_norm = math.sqrt(sum(x**2 for x in b_new))
        b_new = [x / b_new_norm for x in b_new]

        if sum((b_new[i] - b[i])**2 for i in range(n)) < tol:
            break

    b = b_new

    eigenvalue = sum(b[i] * sum(A[i][j] * b[j] for j in range(n)) for i in range(n))
    return eigenvalue

def is_connected(graph):
    n = len(graph)
    visited = [False] * n
    stack = [0]
    visited[0] = True

    while stack:
        node = stack.pop()
        for neighbor in graph[node]:
            if not visited[neighbor]:
                visited[neighbor] = True
                stack.append(neighbor)

    return all(visited)

def generate_regular_graph(n, d):
    if d >= n:
        raise ValueError("Degree must be less than number of vertices")

    while True:
        graph = defaultdict(list)
        edges = []

        for i in range(n):
            for j in range(i + 1, n):
                edges.append((i, j))

        random.shuffle(edges)

        for i in range(n):
            graph[i] = []

```

```

        for i in range(n):
            for j in range(d // 2):
                u, v = edges.pop()
                graph[u].append(v)
                graph[v].append(u)

    if is_connected(graph):
        return graph

def generate_glued_graph(n):
    g1 = generate_regular_graph(n // 2, 3)
    g2 = generate_regular_graph(n // 2, 3)

    graph = defaultdict(list)
    for i in range(n // 2):
        graph[i] = g1[i]
    for i in range(n // 2, n):
        graph[i] = [x + n // 2 for x in g2[i - n // 2]]

    graph[0].append(n // 2)
    graph[n // 2].append(0)

    return graph

def generate_cycle(n):
    graph = defaultdict(list)
    for i in range(n):
        graph[i].append((i + 1) % n)
        graph[i].append((i - 1) % n)
    return graph

def build_non_backtracking_matrix(graph):
    n = len(graph)
    d = len(graph[0])
    m = n * d

    B = [[0 for _ in range(m)] for _ in range(m)]

    for v in range(n):
        for i in range(d):
            u = graph[v][i]
            for j in range(d):
                if graph[u][j] != v:
                    B[v * d + i][u * d + j] = 1

    return B

def compute_nu(graph, B):
    n = len(graph)
    d = len(graph[0])
    rho = power_iteration(B)
    nu = n * max(0, math.log((d - 1) / rho))
    return nu

def generate_tseitin_cnf(graph, sigma):

```

```

n = len(graph)
d = len(graph[0])
cnf = []

for v in range(n):
    for i in range(d):
        u = graph[v][i]
        if u > v:
            cnf.append([(v, i), (u, i), (v, (i + 1) % d), (u, (i + 1) % d)])

for v in range(n):
    for i in range(d):
        cnf.append([(v, i), (v, (i + 1) % d)])

return cnf

def count_dp11(cnf):
    n = len(cnf)
    visited = set()

    def dfs(assignment):
        if len(assignment) == n:
            return 1

        for clause in cnf:
            satisfied = False
            for lit in clause:
                if lit in assignment:
                    satisfied = True
                    break
            if not satisfied:
                return 0

        count = 0
        for lit in cnf[len(assignment)]:
            if lit not in assignment:
                new_assignment = assignment + (lit,)
                if new_assignment not in visited:
                    visited.add(new_assignment)
                    count += dfs(new_assignment)

        return count

    return dfs(tuple())

def run_trial(seed):
    random.seed(seed)
    n_values = [6, 8, 10, 12, 14, 16]
    metric_values = []
    instances_tested = 0
    conjecture_holds = True
    counterexample = ""

    for n in n_values:
        for _ in range(5):
            graph = generate_regular_graph(n, 3)
            B = build_non_backtracking_matrix(graph)

```

```

    nu = compute_nu(graph, B)
    sigma = random.randint(1, n)
    cnf = generate_tseitin_cnf(graph, sigma)
    R = count_dpll(cnf)
    log2_R = math.log2(R) if R > 0 else 0
    metric_values.append(log2_R)
    instances_tested += 1

    if log2_R < 0.1 * nu and nu >= 0.5 * n:
        conjecture_holds = False
        counterexample = f"Found counterexample with n={n}, nu={nu}, log2_R={log2_R}"

for _ in range(2):
    graph = generate_glued_graph(n)
    B = build_non_backtracking_matrix(graph)
    nu = compute_nu(graph, B)
    sigma = random.randint(1, n)
    cnf = generate_tseitin_cnf(graph, sigma)
    R = count_dpll(cnf)
    log2_R = math.log2(R) if R > 0 else 0
    metric_values.append(log2_R)
    instances_tested += 1

    if nu >= 1.5 or log2_R >= 1.5 * n:
        conjecture_holds = False
        counterexample = f"Found counterexample with n={n}, nu={nu}, log2_R={log2_R}"

for _ in range(1):
    graph = generate_cycle(n)
    B = build_non_backtracking_matrix(graph)
    nu = compute_nu(graph, B)
    sigma = random.randint(1, n)
    cnf = generate_tseitin_cnf(graph, sigma)
    R = count_dpll(cnf)
    log2_R = math.log2(R) if R > 0 else 0
    metric_values.append(log2_R)
    instances_tested += 1

    if nu >= 1.5 or log2_R >= 1.5 * n:
        conjecture_holds = False
        counterexample = f"Found counterexample with n={n}, nu={nu}, log2_R={log2_R}"

mean_metric = sum(metric_values) / len(metric_values)
std_metric = math.sqrt(sum((x - mean_metric) ** 2 for x in metric_values) / len(metric_values))

return {
    "metric_name": "log2_R",
    "metric_value": mean_metric,
    "instances_tested": instances_tested,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample,
    "std_metric": std_metric
}

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] if len(sys.argv) > 1 else [2, 3, 5, 7, 11, 13, 17,

```

```

results = []
for seed in seeds:
    result = run_trial(seed)
    print(f"TRIAL: {result}")
    results.append(result)

mean_metric = sum(r["metric_value"] for r in results) / len(results)
std_metric = math.sqrt(sum((r["metric_value"] - mean_metric) ** 2 for r in results) / len(results))
support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

if all(r["conjecture_holds"] for r in results):
    print(f"RESULT: SUPPORTED mean={mean_metric} std={std_metric} support_fraction={support_fraction}")
elif any(not r["conjecture_holds"] for r in results):
    first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
    counterexample = next(r["counterexample"] for r in results if not r["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={first_failing_seed}")
else:
    print("RESULT: INCONCLUSIVE reason=unknown")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_b2cd758f.py", line 257, in <module>
    result = run_trial(seed)
              ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_b2cd758f.py", line 197, in run_trial
    B = build_non_backtracking_matrix(graph)
        ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_b2cd758f.py", line 129, in build_non_backtracking_matrix
    if graph[u][j] != v:
        ~~~~~^
IndexError: list index out of range

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed with an IndexError in build_non_backtracking_matrix before any data was produced, so neither the support nor the falsification condition can be evaluated. | next: Fix the indexing bug in build_non_backtracking_matrix (likely an off-by-one or assumption about graph[u]'s neighbor list length) and re-run the pre-registered protocol.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	119939
2	preregistration	claude_max	opus	0	0	7717
3	novelty	claude_max	opus	0	0	3442
4	novelty	claude_max	opus	0	0	8121
5	test_gen	mistral	codestral-latest	0	0	325594
6	test_gen	mistral	codestral-latest	0	0	320417
7	test_gen	mistral	codestral-latest	0	0	318573
8	test_gen	mistral	codestral-latest	0	0	313863
9	judge	claude_max	opus	0	0	3920

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 1421586 ms total latency. Provider mix: {'claude_max': 5, 'mistral': 4}

(full prompt+response transcripts available in `research/audit/a58cbdc6e05.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/a58cbdc6e05.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/a58cbdc6e05.tar.gz` (if generated)